

今日核心:

1. 综合案例：数据的增删改查（CRUD）
2. Promise

1. [图片素材.zip](#)

## 案例-我的书架

今天首先来完成一个综合案例-我的数据，重点练习实际开发中对于数据的增删改查操作

[文档传送门](#)

|           |  |      |
|-----------|--|------|
| 欢迎使用      |  |      |
| 开心一笑      |  |      |
| 随机笑话      |  | GET  |
| 若干条随机笑话   |  | GET  |
| 新闻接口      |  |      |
| 获取-新闻列表   |  | GET  |
| 省市区       |  |      |
| 获取-省份列表   |  | GET  |
| 获取-城市列表   |  | GET  |
| 获取-地区列表   |  | GET  |
| 注册和登录     |  |      |
| 注册账号-form |  | POST |
| 登录-form   |  | POST |
| 注册账号-JSON |  | POST |

注册账号-JSON

POST

登录-JSON

POST

验证码获取

GET

验证码登录

POST

## 图书管理



获取-图书列表

GET

新增-图书

POST

删除-图书

DELETE

获取-图书详情

GET

修改-图书详情

PUT

黑马程序员



7:52

100



## 我的书架



书名：《西游记》

作者：吴承恩

出版社：人民文学出版社



书名：《三国演义》

作者：罗贯中

出版社：人民文学出版社

删除



书名：《水浒传》

作者：施耐庵

出版社：人民文学出版社

7:51

100%

## < 新增图书

图书姓名: 请输入图书名字

图书作者: 请输入图书作者

图书出版社: 请输入图书出版社

保存

黑马程序员

7:52

! 100

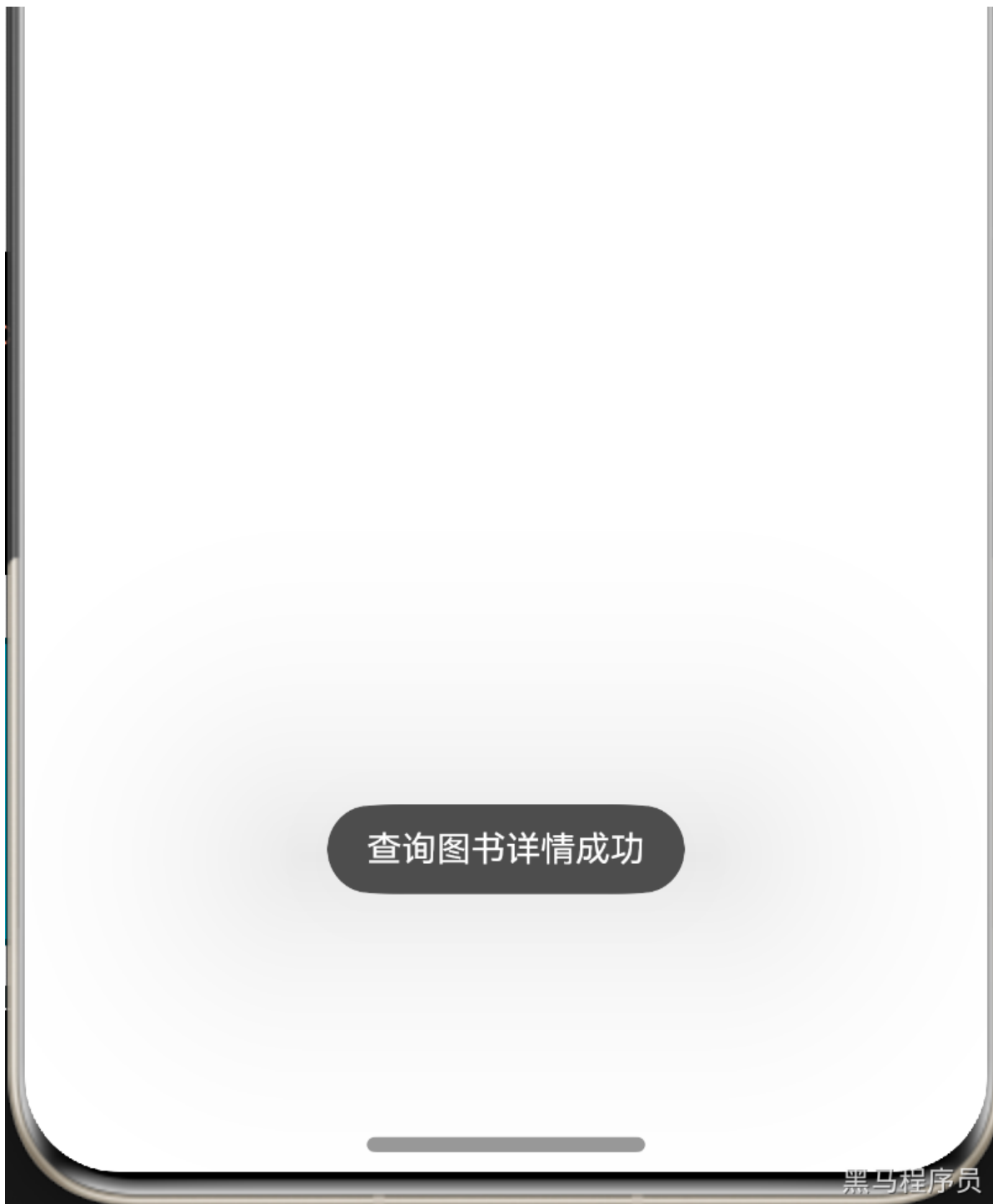
## < 修改图书

图书姓名: 《三国演义》

图书作者: 罗贯中

图书出版社: 人民文学出版社

保存



黑马程序员

## 1. 获取图书

---

:::warning

核心步骤:

1. 获取图书数据
  1. aboutToAppear
    1. http模块获取图书
2. 渲染到页面上
  1. 数组-ForEach
3. 增加动画效果
  1. if(数组长度为 0)没有数据时: 展示 loading
  2. 获取数据之后: 隐藏 loading

### 3. 找到动画的组件

...

```
1  import { router } from '@kit.ArkUI'
2
3
4
5
6  export interface Book {
7    id: number
8    bookname: string
9    author: string
10   publisher: string
11 }
12
13 export const creator: string = 'itheima'
14
15 @Entry
16 @Component
17 struct Day02_01_BookShelf {
18   creator: string = creator
19   @State books: Book[] = [{
20     "id": 366351,
21     "bookname": "《西游记》",
22     "author": "吴承恩",
23     "publisher": "人民文学出版社"
24   }]
25   @State isLoading: boolean = true
26
27   build() {
28     Column() {
29       // 头部
30       this.HeaderBuilder()
31       LoadingProgress()
32         .width(50)
33
34       List({ space: 15 }) {
35         ForEach(this.books, (item: Book) => {
36           ListItem() {
37             bookItem({ data: item })
38           }
39           .swipeAction({
40             end: () => {
41               this.itemEnd(item)
42             },
43             edgeEffect: SwipeEdgeEffect.Spring
44           })
45           .onClick(() => {
46
47             })
48         })
49       }
50       .padding(20)
51     }
52     .height('100%')
```

```

53     .width('100%')
54 }
55
56 @Builder
57 HeaderBuilder() {
58     Row() {
59         Image($r('app.media.ic_public_drawer_filled'))
60         .width(20);
61
62         Text('我的书架')
63         .fontSize(25)
64
65         Image($r('app.media.ic_public_add'))
66         .width(20)
67         .onClick(() => {
68             router.pushUrl({
69                 url: 'pages/Day02_01_BookShelf_Add'
70             })
71         })
72     }
73     .width('100%')
74     .justifyContent(FlexAlign.SpaceBetween)
75     .height(60)
76     .padding(10)
77     .border({ width: { bottom: 2 }, color: '#f0f0f0' })
78     .backgroundColor(Color.White)
79 }
80
81 @Builder
82 itemEnd(item: Book) {
83     Row() {
84         Button('删除')
85         .type(ButtonType.Normal)
86         .backgroundColor('#da3231')
87         .onClick(() => {
88             AlertDialog.show({ message: '点了删除' })
89         })
90         .height('100%')
91     }
92
93 }
94 }
95
96 @Component
97 struct bookItem {
98     data: Partial<Book> = {}
99
100     build() {
101         Row({ space: 10 }) {
102             Image($r('app.media.ic_public_cover'))
103             .width(108)
104             .height(108)
105             Column({ space: 5 }) {
106
107                 Text('书名: ' + this.data.bookname)

```

```

108         .fontSize(20)
109         Text('作者: ' + this.data.author)
110         .fontSize(14)
111         .fontColor(Color.Gray)
112         Blank()
113         Text('出版社: ' + this.data.publisher)
114         .fontSize(14)
115         .fontColor(Color.Gray)
116     }
117     .padding({ top: 10, bottom: 10 })
118     .height(108)
119     .alignItems(HorizontalAlign.Start)
120 }
121 }
122 }

```

Fence 1

```

1  import { curves, router } from '@kit.ArkUI'
2  import http from '@ohos.net.http'
3
4
5  interface BookResponse {
6      message: string
7      data: Book[]
8  }
9
10 export interface Book {
11     id: number
12     bookname: string
13     author: string
14     publisher: string
15 }
16
17 export const creator: string = 'itheima'
18
19 @Entry
20 @Component
21 struct Day02_01_BookShelf {
22     creator: string = creator
23     @State books: Book[] = [{
24         "id": 366351,
25         "bookname": "《西游记》",
26         "author": "吴承恩",
27         "publisher": "人民文学出版社"
28     }]
29     @State isLoading: boolean = true
30     req: http.HttpRequest = http.createHttp()
31
32     getData(showLoading: boolean = false) {
33         if (showLoading) this.isLoading = true
34         this.req.request(`https://hmajax.itheima.net/api/books?
creator=${this.creator}`)
35         .then(res => {
36             const bookResult = JSON.parse(res.result.toString()) as
BookResponse

```

```

37         this.books = bookResult.data
38         if (showLoading) this.isLoading = false
39     })
40 }
41
42 onPageShow(): void {
43     this.getData(true)
44 }
45
46 build() {
47     column() {
48         // 头部
49         this.HeaderBuilder()
50         LoadingProgress()
51         .width(50)
52         .visibility(this.isLoading === true ? visibility.Visible :
visibility.None)
53         .animation({ curve: curves.springMotion() })
54
55
56         List({ space: 15 }) {
57             ForEach(this.books, (item: Book) => {
58                 ListItem() {
59                     bookItem({ data: item })
60                 }
61                 .swipeAction({
62                     end: () => {
63                         this.itemEnd(item)
64                     },
65                     edgeEffect: SwipeEdgeEffect.Spring
66                 })
67                 .onClick(() => {
68
69                 })
70             })
71         }
72         .visibility(this.isLoading === false ? visibility.Visible :
visibility.None)
73         .animation({ curve: curves.springMotion() })
74         .padding(20)
75     }
76     .height('100%')
77     .width('100%')
78 }
79
80 @Builder
81 HeaderBuilder() {
82     Row() {
83         Image($r('app.media.ic_public_drawer_filled'))
84         .width(20);
85
86         Text('我的书架')
87         .fontSize(25)
88
89         Image($r('app.media.ic_public_add'))

```

```

90         .width(20)
91         .onClick(() => {
92             router.pushUrl({
93                 url: 'pages/Day02_01_BookShelf_Add'
94             })
95         })
96     }
97     .width('100%')
98     .justifyContent(FlexAlign.SpaceBetween)
99     .height(60)
100    .padding(10)
101    .border({ width: { bottom: 2 }, color: '#f0f0f0' })
102    .backgroundColor(Color.White)
103 }
104
105 @Builder
106 itemEnd(item: Book) {
107     Row() {
108         Button('删除')
109             .type(ButtonType.Normal)
110             .backgroundColor('#da3231')
111             .onClick(() => {
112                 AlertDialog.show({ message: '点了删除' })
113             })
114             .height('100%')
115     }
116 }
117 }
118 }
119
120 @Component
121 struct bookItem {
122     data: Partial<Book> = {}
123
124     build() {
125         Row({ space: 10 }) {
126             Image($r('app.media.ic_public_cover'))
127                 .width(108)
128                 .height(108)
129             Column({ space: 5 }) {
130
131                 Text('书名: ' + this.data.bookname)
132                     .fontSize(20)
133                 Text('作者: ' + this.data.author)
134                     .fontSize(14)
135                     .fontColor(Color.Gray)
136                 Blank()
137                 Text('出版社: ' + this.data.publisher)
138                     .fontSize(14)
139                     .fontColor(Color.Gray)
140             }
141             .padding({ top: 10, bottom: 10 })
142             .height(108)
143             .alignItems(HorizontalAlign.Start)
144         }

```

```
145     }
146 }
```

Fence 2

## 2. 新增图书

:::warning

**核心步骤:**

1. 点击提交数据，非空判断
  1. 为空：promptAction提示
  2. 不为空：
    1. 提交，除了输入的数据以外， creator
    2. 为了便于复用，
      1. 首页：creator 提取为变量，导出
      2. 添加页：导入并使用
2. 提示用户promptAction，并返回上一页
  1. router.back()
3. 首页：
  1. aboutToAppear 页面不销毁不会重新执行
  2. 调整获取图书的逻辑到 onPageShow 中
  - ...

```
1  @Entry
2  @Component
3  struct Bookshelf_Add {
4      @State bookname: string = ''
5      @State author: string = ''
6      @State publisher: string = ''
7
8      build() {
9          Navigation() {
10             Column({ space: 10 }) {
11                 Row() {
12                     Text('图书姓名:')
13                     TextInput({
14                         placeholder: '请输入图书名字',
15                         text: $$this.bookname
16                     })
17                     .height(30)
18                     .backgroundColor(Color.Transparent)
19                     .layoutweight(1)
20                     .padding({ left: 10, top: 0, bottom: 0 })
21                 }
22
23                 Divider()
24                 Row() {
25                     Text('图书作者:')
26                     TextInput({
27                         placeholder: '请输入图书作者',
28                         text: $$this.author
```

```

29         })
30         .height(30)
31         .backgroundColor(Color.Transparent)
32         .layoutWeight(1)
33         .padding({ left: 10, top: 0, bottom: 0 })
34     }
35
36     Divider()
37     Row() {
38         Text('图书出版社:')
39         TextInput({
40             placeholder: '请输入图书出版社',
41             text: $$this.publisher
42         })
43         .height(30)
44         .backgroundColor(Color.Transparent)
45         .layoutWeight(1)
46         .padding({ left: 10, top: 0, bottom: 0 })
47     }
48
49     Divider()
50
51
52     Button('保存')
53         .width('100%')
54         .margin({ top: 20 })
55         .type(ButtonType.Normal)
56         .borderRadius(10)
57         .onClick(() => {
58             })
59
60     }
61     .padding(20)
62 }
63 .title('新增图书')
64 .titleMode(NavigationTitleMode.Mini)
65 .backButtonIcon($r('app.media.ic_public_arrow_left'))
66 }
67 }

```

Fence 3

```

1  import { creator } from '../data/BookShelf_Constants'
2  import http from '@ohos.net.http'
3  import { Book } from './Day02_01_BookShelf_Home'
4  import { promptAction, router } from '@kit.ArkUI'
5
6  interface AddResponse {
7      message: string
8      data: Book
9  }
10
11  @Entry
12  @Component
13  struct Day02_01_BookShelf_Add {
14      @State bookname: string = ''

```

```
15 @State author: string = ''
16 @State publisher: string = ''
17 creator: string = creator
18
19 build() {
20     Navigation() {
21         Column({ space: 10 }) {
22             Row() {
23                 Text('图书姓名:')
24                 TextInput({
25                     placeholder: '请输入图书名字',
26                     text: $$this.bookname
27                 })
28                 .height(30)
29                 .backgroundColor(Color.Transparent)
30                 .layoutweight(1)
31                 .padding({ left: 10, top: 0, bottom: 0 })
32             }
33
34             Divider()
35             Row() {
36                 Text('图书作者:')
37                 TextInput({
38                     placeholder: '请输入图书作者',
39                     text: $$this.author
40                 })
41                 .height(30)
42                 .backgroundColor(Color.Transparent)
43                 .layoutweight(1)
44                 .padding({ left: 10, top: 0, bottom: 0 })
45             }
46
47             Divider()
48             Row() {
49                 Text('图书出版社:')
50                 TextInput({
51                     placeholder: '请输入图书出版社',
52                     text: $$this.publisher
53                 })
54                 .height(30)
55                 .backgroundColor(Color.Transparent)
56                 .layoutweight(1)
57                 .padding({ left: 10, top: 0, bottom: 0 })
58             }
59
60             Divider()
61
62
63             Button('保存')
64                 .width('100%')
65                 .margin({ top: 20 })
66                 .type(ButtonType.Normal)
67                 .borderRadius(10)
68                 .onClick(() => {
69                     const req = http.createHttp()
```

```

70         req.request('https://hmajax.itheima.net/api/books', {
71             method: http.RequestMethod.POST,
72             header: {
73                 contentType: 'application/json'
74             },
75             extraData: {
76                 bookname: this.bookname,
77                 author: this.author,
78                 publisher: this.publisher,
79                 creator: this.creator
80             }
81         })
82         .then(res => {
83             const addRes = JSON.parse(res.result.toString()) as
AddResponse
84             promptAction.showToast({ message: addRes.message })
85
86             router.back()
87         })
88     })
89
90     }
91     .padding(20)
92 }
93 .title('新增图书')
94 .titleMode(NavigationTitleMode.Mini)
95 .backButtonIcon($r('app.media.ic_public_arrow_left'))
96 }
97 }

```

Fence 4

### 3. 删除图书

:::warning

**核心步骤:**

1. 侧滑弹框提示用户，确认调用删除接口
  1. 弹框参考 `promptAction.showDialog`文档使用
  2. 确认：删除图书

# 删除-图书

一般用于删除数据

DELETE

https://hmajax.itheima.net/api/books/{id}

▶ 调试

method中

删除-某本图书

https://hmajax.itheima.net/api/books/10

https://hmajax.itheima.net/api/books/20

1.

请求参数

Path 参数

id number 图书id 必需

示例值: 1

黑马程序员

3. 取消: 啥都不干

2. 删除成功之后重新获取图书数据

1. 调用获取数据的方法 this.getBooks

...

```

1  import { curves, router } from '@kit.ArkUI'
2  import http from '@ohos.net.http'
3
4
5  interface BookResponse {
6      message: string
7      data: Book[]
8  }
9
10 export interface Book {
11     id: number
12     bookname: string
13     author: string
14     publisher: string
15 }
16
17 export const creator: string = 'itheima'
18
19 @Entry
20 @Component
21 struct Day02_01_BookShelf {
22     creator: string = creator
23     @State books: Book[] = [{
24         "id": 366351,
25         "bookname": "《西游记》",
26         "author": "吴承恩",
27         "publisher": "人民文学出版社"
28     }]
29     @State isLoading: boolean = true
30     req: http.HttpRequest = http.createHttp()
31
32     getData(showLoading: boolean = false) {
33         if (showLoading) this.isLoading = true

```

```

34     this.req.request(`https://hmajax.itheima.net/api/books?
creator=${this.creator}`)
35     .then(res => {
36         const bookResult = JSON.parse(res.result.toString()) as
BookResponse
37         this.books = bookResult.data
38         if (showLoading) this.isLoading = false
39     })
40 }
41
42 onPageShow(): void {
43     this.getData(true)
44 }
45
46 build() {
47     Column() {
48         // 头部
49         this.HeaderBuilder()
50         LoadingProgress()
51         .width(50)
52         .visibility(this.isLoading === true ? visibility.Visible :
visibility.None)
53         .animation({ curve: curves.springMotion() })
54
55
56         List({ space: 15 }) {
57             ForEach(this.books, (item: Book) => {
58                 ListItem() {
59                     bookItem({ data: item })
60                 }
61                 .swipeAction({
62                     end: () => {
63                         this.itemEnd(item)
64                     },
65                     edgeEffect: SwipeEdgeEffect.Spring
66                 })
67                 .onClick(() => {
68
69                 })
70             })
71         }
72         .visibility(this.isLoading === false ? visibility.Visible :
visibility.None)
73         .animation({ curve: curves.springMotion() })
74         .padding(20)
75     }
76     .height('100%')
77     .width('100%')
78 }
79
80 @Builder
81 HeaderBuilder() {
82     Row() {
83         Image($r('app.media.ic_public_drawer_filled'))
84         .width(20);

```

```

85
86     Text('我的书架')
87         .fontSize(25)
88
89     Image($r('app.media.ic_public_add'))
90         .width(20)
91         .onClick(() => {
92             router.pushUrl({
93                 url: 'pages/Day02_01_BookShelf_Add'
94             })
95         })
96     }
97     .width('100%')
98     .justifyContent(FlexAlign.SpaceBetween)
99     .height(60)
100    .padding(10)
101    .border({ width: { bottom: 2 }, color: '#f0f0f0' })
102    .backgroundColor(Color.White)
103 }
104
105 @Builder
106 itemEnd(item: Book) {
107     Row() {
108         Button('删除')
109             .type(ButtonType.Normal)
110             .backgroundColor('#da3231')
111             .onClick(() => {
112                 promptAction.showDialog({
113                     title: '提示',
114                     message: `确认删除图书${item.bookname}?`,
115                     buttons: [
116                         {
117                             text: '取消',
118                             color: '#000'
119                         },
120                         {
121                             text: '确认',
122                             color: '#367bf6'
123                         }
124                     ]
125                 })
126                 .then(res => {
127                     if (res.index === 1) {
128                         // 删除
129
130                     this.req.request(`https://hmajax.itheima.net/api/books/${item.id}`,
131                         {
132                             method: http.RequestMethod.DELETE
133                         })
134                     .then(res => {
135                         promptAction.showToast({ message: '删除成功' })
136                         this.getData()
137                     })
138                 })

```

```

139         })
140
141     })
142     .height('100%')
143 }
144
145 }
146 }
147
148 @Component
149 struct bookItem {
150     data: Partial<Book> = {}
151
152     build() {
153         Row({ space: 10 }) {
154             Image($r('app.media.ic_public_cover'))
155                 .width(108)
156                 .height(108)
157             Column({ space: 5 }) {
158
159                 Text('书名: ' + this.data.bookname)
160                     .fontSize(20)
161                 Text('作者: ' + this.data.author)
162                     .fontSize(14)
163                     .fontColor(Color.Gray)
164                 Blank()
165                 Text('出版社: ' + this.data.publisher)
166                     .fontSize(14)
167                     .fontColor(Color.Gray)
168             }
169             .padding({ top: 10, bottom: 10 })
170             .height(108)
171             .alignItems(HorizontalAlign.Start)
172         }
173     }
174 }

```

Fence 5

## 4. 全部删除

点击全部删除

:::success

**核心步骤:**

1. 找个全部删除按钮，比如顶部的左侧
2. 提示用户
  1. 确认：全部删除
    1. 循环图书数组
    2. 获取每一项的 id
    3. 依次调用删除接口
    4. 删除完毕之后，重新获取数据

:::

## 5. 修改图书

---



7:52

100



## 我的书架



书名：《西游记》

作者：吴承恩

出版社：人民文学出版社



书名：《三国演义》

作者：罗贯中

出版社：人民文学出版社

删除



书名：《水浒传》

作者：施耐庵

出版社：人民文学出版社

7:52

! 100

## < 修改图书

图书姓名: 《三国演义》

图书作者: 罗贯中

图书出版社: 人民文学出版社

保存



:::warning

#### 核心步骤:

1. **首页:** 点击携带 id 跳转到编辑页
2. **编辑页:**
  1. 根据 id 查询图书并渲染到页面上
  2. 修改之后点击保存, 保存成功之后提示用户, 并返回首页

...

```
1  @Entry
2  @Component
3  struct BookShelf_Edit {
4      @State bookname: string = ''
5      @State author: string = ''
6      @State publisher: string = ''
7
8      aboutToAppear(): void {
9
10     }
11
12     build() {
13         Navigation() {
14             Column({ space: 10 }) {
15                 Row() {
16                     Text('图书姓名:')
17                     TextInput({
18                         placeholder: '请输入图书名字',
```

```

19         text: $$this.bookname
20     })
21     .height(30)
22     .backgroundColor(Color.Transparent)
23     .layoutWeight(1)
24     .padding({ left: 10, top: 0, bottom: 0 })
25 }
26
27 Divider()
28 Row() {
29     Text('图书作者:')
30     TextInput({
31         placeholder: '请输入图书作者',
32         text: $$this.author
33     })
34     .height(30)
35     .backgroundColor(Color.Transparent)
36     .layoutWeight(1)
37     .padding({ left: 10, top: 0, bottom: 0 })
38 }
39
40 Divider()
41 Row() {
42     Text('图书出版社:')
43     TextInput({
44         placeholder: '请输入图书出版社',
45         text: $$this.publisher
46     })
47     .height(30)
48     .backgroundColor(Color.Transparent)
49     .layoutWeight(1)
50     .padding({ left: 10, top: 0, bottom: 0 })
51 }
52
53 Divider()
54
55
56 Button('保存')
57     .width('100%')
58     .margin({ top: 20 })
59     .type(ButtonType.Normal)
60     .borderRadius(10)
61     .onClick(() => {
62     })
63
64 }
65 .padding(20)
66 }
67 .title('修改图书')
68 .titleMode(NavigationTitleMode.Mini)
69 .backButtonIcon($r('app.media.ic_public_arrow_left'))
70 }
71 }

```

```
1 import http from '@ohos.net.http'
2 import { Book } from './Day02_01_BookShelf_Home'
3 import { promptAction, router } from '@kit.ArkUI'
4 import { creator } from '../data/BookShelf_Constants'
5
6 interface GetBookResponse {
7     message: string
8     data: Book
9 }
10
11 interface BookInfo {
12     id: number
13 }
14
15 interface EditBookResponse {
16     message: string
17     data: Book
18 }
19
20 @Entry
21 @Component
22 struct Day02_01_BookShelf_Add {
23     @State bookname: string = ''
24     @State author: string = ''
25     @State publisher: string = ''
26     bookId: number = 0
27     req: http.HttpRequest = http.createHttp()
28
29     aboutToAppear(): void {
30         // 根据 id 获取详情
31         this.bookId = (router.getParams() as BookInfo).id
32
33         // 获取详情
34         this.req.request(`https://hmajax.itheima.net/api/books/${this.bookId}`)
35             .then(res => {
36                 const bookRes = JSON.parse(res.result.toString()) as
37                 GetBookResponse
38                 promptAction.showToast({
39                     message: bookRes.message
40                 })
41                 this.bookname = bookRes.data.bookname
42                 this.author = bookRes.data.author
43                 this.publisher = bookRes.data.publisher
44             })
45     }
46
47     build() {
48         Navigation() {
49             Column({ space: 10 }) {
50                 Row() {
51                     Text('图书姓名:')
52                     TextInput({
53                         placeholder: '请输入图书名字',
54                         text: $$this.bookname
```

```

55         .height(30)
56         .backgroundColor(Color.Transparent)
57         .layoutweight(1)
58         .padding({ left: 10, top: 0, bottom: 0 })
59     }
60
61     Divider()
62     Row() {
63         Text('图书作者:')
64         TextInput({
65             placeholder: '请输入图书作者',
66             text: $$this.author
67         })
68         .height(30)
69         .backgroundColor(Color.Transparent)
70         .layoutweight(1)
71         .padding({ left: 10, top: 0, bottom: 0 })
72     }
73
74     Divider()
75     Row() {
76         Text('图书出版社:')
77         TextInput({
78             placeholder: '请输入图书出版社',
79             text: $$this.publisher
80         })
81         .height(30)
82         .backgroundColor(Color.Transparent)
83         .layoutweight(1)
84         .padding({ left: 10, top: 0, bottom: 0 })
85     }
86
87     Divider()
88
89
90     Button('保存')
91         .width('100%')
92         .margin({ top: 20 })
93         .type(ButtonType.Normal)
94         .borderRadius(10)
95         .onClick(() => {
96             AlertDialog.show({
97                 message: JSON.stringify({
98                     bookname: this.bookname,
99                     author: this.author,
100                     publisher: this.publisher,
101                     creator: creator
102                 })
103             })
104
105             this.req.request(`https://hmajax.itheima.net/api/books/${this.bookId}`, {
106                 method: http.RequestMethod.PUT,
107                 header: {
108                     contentType: 'application/json'
109                 },

```

```

109         extraData: {
110             bookname: this.bookname,
111             author: this.author,
112             publisher: this.publisher,
113         }
114     })
115     .then(res => {
116         const editRes = JSON.parse(res.result.toString()) as
EditBookResponse
117         promptAction.showToast({
118             message: editRes.message
119         })
120         setTimeout(() => {
121             router.back()
122         }, 1000)
123     })
124
125     })
126
127     }
128     .padding(20)
129 }
130 .title('修改图书')
131 .titleMode(NavigationTitleMode.Mini)
132 .backButtonIcon($r('app.media.ic_public_arrow_left'))
133 }
134 }

```

# 同步代码&异步代码

Promise可以用来更好的管理异步代码，与异步代码相对的是同步代码，咱们先来认识一下如何区分

**同步代码：**逐行执行，需原地等待结果后，才继续向下执行

**异步代码：**调用后**耗时**，不阻塞代码继续执行，将来完成后，触发回调函数传递结果

**划重点：**异步代码的结果，通过 **回调函数** 获取

```
1  /**
2   * 1. 同步代码
3   * */
4  console.log(1)
5  const num = 1 + 1
6  console.log(num)
7
8  /**
9   * 2. 同步+异步（定时器）
10  * */
11 console.log(1)
12 setTimeout(() => {
13   console.log(2)
14 }, 1000)
15 console.log(3)
16
17 /**
18  * 3. 同步+异步（网络请求）
19  * */
20 console.log('1')
21
22 const req = http.createHttp()
23 req.request('https://api-vue-base.itheima.net/api/joke')
24   .then((res: http.HttpResponse) => {
25     console.log('3')
26   })
27
28 console.log('2')
29
```

# Promise

setTimeout 和 http 的 request 都是异步的操作，为什么第二个是在 then 里面传递回调函数呢？这是因为 he 返回的是一个 Promise 对象，后续课程中咱们会经常碰到.then的写法，为了更好的用这种写法组织异步代码，咱们来一起学习一下 Promise

## 1. 什么是 Promise

⚠️

Promise 是一种用于处理异步操作的对象，可以将异步操作转换为类似于同步操作的风格，以方便代码编写和维护

**简而言之：**Promise 用来管理异步，方便编码

⋮

实际开发中如果异步操作作用到了 Promise 来进行管理那么：

1. 那么获取异步操作结果（成功 or 失败）的方式都是一样的
2. 能够解决回调函数地狱（多层回调函数嵌套）

如何用 Promise 来管理异步任务呢？咱们分 3 步走：

1. 内部执行异步代码
2. 传递成功结果
3. 传递失败结果

```
1 // 实例化 Promise 对象
2 const p = new Promise<string>(() => {
3   // 执行任意代码，主要是异步
4   setTimeout(() => {
5     // 比如：获取随机数
6     const randomNum = Math.floor(Math.random() * 100)
7     console.log('随机数是:', randomNum)
8   })
9 })
10 }
```

Fence 9

```
1 const p = new Promise<string>((resolve) => {
2   // 执行任意代码，主要是异步
3   setTimeout(() => {
4     // 比如：获取随机数
5     const randomNum = Math.floor(Math.random() * 100)
6     resolve(randomNum.toString())
7   })
8 })
9
10 p.then(res => {
11   console.log('res:', res)
12 })
```

Fence 10

```
1 const p = new Promise<string>((resolve, reject) => {
2   // 执行任意代码，主要是异步
3   setTimeout(() => {
```

```

4      // 比如: 获取随机数
5      const randomNum = Math.floor(Math.random() * 100)
6      // resolve(randomNum.toString())
7      reject(randomNum.toString())
8    })
9  })
10
11  p.then(res => {
12    console.log('res:', res)
13  }, (err:string)=>{
14    console.log('err:', err)
15  })

```

Fence 11

提取一下核心代码:

```

1  const p = new Promise<string>((resolve, reject) => {
2    // 执行任意代码, 主要是异步
3    // 成功 resolve(成功结果) then 执行
4    // 失败 reject(失败结果) then第二个回调函数或 catch 执行
5  })
6
7  p.then(res => {}, (err: string) => {})
8  // 或者
9  p.then(res=>{}).catch((err:string)=>{})

```

Fence 12

:::warning

再来看看http模块的 request 方法,虽然无法看到源码, 但是内部大概做的是这样的事情

:::

```

1  request():Promise<string>{
2    return new Promise<string>((resolve, reject)=>{
3      // 发送网络请求
4      // 解析结果 resolve(解析之后的结果)
5    })
6  }

```

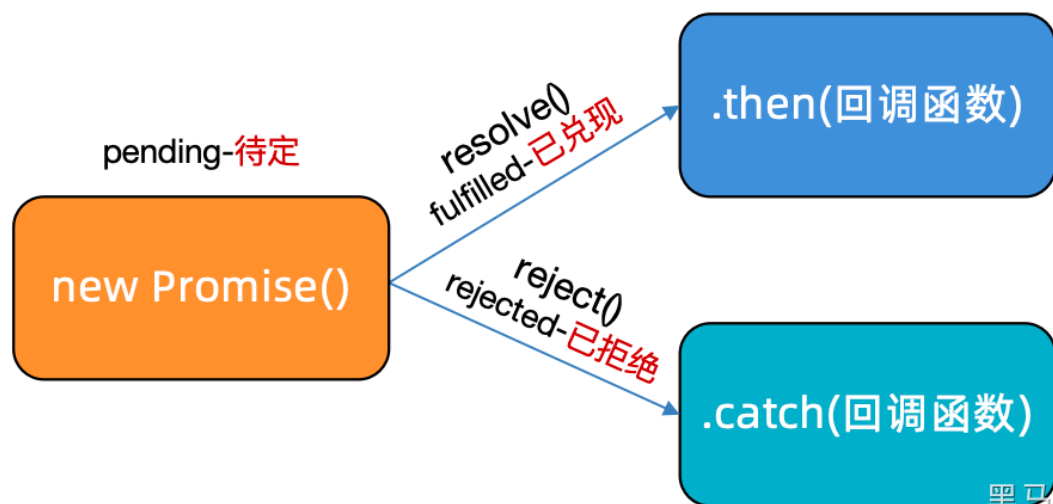
Fence 13

## 2. Promise 的状态

Promise 必然处于 3 种状态中的某一种, 调用resolve,reject 的本质就是更改他的状态

3 种状态:

1. 待定 (pending) : 初始状态, 既没有被兑现, 也没有被拒绝
2. 已兑现 (fulfilled) : 意味着操作成功完成
3. 已拒绝 (rejected) : 意味着操作失败



黑马程序员

:::warning

注意：状态的改变不可逆

调用 resolve 之后再调用 reject，状态还是 已兑现，反之亦然

:::

```

1  const p = new Promise<string>((resolve, reject) => {
2      resolve('success')
3      reject('err')
4  })
5
6  p.then(res => {
7      console.log('res:', res)
8  })
9      .catch((err: string) => {
10         console.log('err:', err)
11     })

```

Fence 14

### 3. 回调函数地狱

如果回调函数一直【嵌套】下去，代码的可读性会非常糟糕

一般在多个异步操作【彼此依赖】的时候会出现回调函数嵌套的情况，（c 依赖 b，b 依赖 a），比如：

|         |     |     |
|---------|-----|-----|
| 湖北省     | 广州市 |     |
| 湖南省     | 韶关市 |     |
|         |     |     |
| 广东省     | 深圳市 | 罗湖区 |
|         |     |     |
| 广西壮族自治区 | 珠海市 | 福田区 |
| 海南省     | 汕头市 | 南山区 |

黑马程序员

展示用的组件是:[TextPicker](#)

1. 获取所有省份
2. 获取第一个省的所有城市

### 3. 获取第一个城市的所有地区

这里的数据来源于 3 个接口，【地区】依赖于【城市】、【城市】依赖于【省】，所以代码写起来会是这样的结果

```
1  import http from '@ohos.net.http'
2
3  interface IResponse {
4      message: string
5      list: string[]
6  }
7
8  const req = http.createHttp()
9  let provinces: string[] = []
10 let citys: string[] = []
11 let areas: string[] = []
12 req.request("https://hmajax.itheima.net/api/province")
13   .then(res => {
14       provinces = (JSON.parse(res.result as string) as IResponse).list
15       req.request("https://hmajax.itheima.net/api/city?pname=" +
16 encodeURIComponent(provinces[0])).then(res => {
17         citys = (JSON.parse(res.result as string) as IResponse).list
18         req.request("https://hmajax.itheima.net/api/area?pname=" +
19 encodeURIComponent(provinces[0]) + "&cname=" + encodeURIComponent(citys[0]))
20           .then(res => {
21               areas = (JSON.parse(res.result as string) as IResponse).list
22               console.log('省份', provinces)
23               console.log('城市', citys)
24               console.log('地区', areas)
25           })
26       })
27   })
```

Fence 15

刚刚的代码化简一下:

```
1  promise对象1.then(res1=>{
2      promise对象2.then(res2=>{
3          promise对象3.then(res3=>{
4              promise对象4.then(res4=>{
5                  //.... 可以一直写下去
6              })
7          })
8      })
9  })
```

Fence 16

:::warning

上述写法就叫做回调函数地狱

:::

## 4. 链式编程-基本使用

上一节的写法较为繁琐，这一节咱们通过 Promise 的链式编程来解决这个问题<sup>1</sup>



黑马程序员

:::warning

之所以可以这么写，主要是利用了 Promise 对象的2 个特性:

1. Promise 的 then 方法会返回一个新Promise 对象
2. then 方法的返回值会影响这个 新Promise对象的结果

链式编程写出来的代码结构大概是这样子，then 和 then 之间是平级的

...

```
1  promise对象
2  .then(() => {
3    // 略
4  }).then(()=>{
5    // 略
6  }).then(()=>{
7    // 略
8  })
9
```

Fence 17

```
1  function randomNum(delay: number): Promise<string> {
2    return new Promise<string>((resolve, reject) => {
3      const num = Math.floor(Math.random() * 100)
4      resolve(num.toString())
5    })
6  }
7
8
9  randomNum(1000)
10 .then(res => {
11   console.log('随机数1: ', res)
12   return randomNum(2000)
13 })
14 .then(res => {
15   console.log('随机数2: ', res)
16   return randomNum(3000)
17 })
18 .then(res => {
19   console.log('随机数3: ', res)
20   return randomNum(3000)
21 })
```

Fence 18

## 5. 链式编程-实际应用

使用刚刚学习的链式编程来改写之前那个省市区的案例



黑马程序员

:::warning

核心步骤:

1. 在第一个 then 中返回新的 Promise 对象
2. 在第二个 then 中返回新的 Promise 对象
3. 以此类推
- ...

```

1  req.request('https://hmajax.itheima.net/api/province')
2    .then(res => {
3      const proRes = JSON.parse(res.result.toString()) as IResponse
4      pList = proRes.list
5      // 获取第一个省对应的城市
6      return this.req.request('https://hmajax.itheima.net/api/city?pname=' +
7        encodeURIComponent('广东省'))
8    })
9    .then(res => {
10     const cityRes = JSON.parse(res.result.toString()) as IResponse
11     cList = cityRes.list
12     // url?key=value&key2=value2 格式不要写错啦
13     return this.req.request(`https://hmajax.itheima.net/api/area?
14       pname=${encodeURIComponent('广东省')}&cname=${encodeURIComponent('深圳市')}`)
15   })
16   .then(res => {
17     const areaRes = JSON.parse(res.result.toString()) as IResponse
18     aList = areaRes.list
19     // 设置给 组件的数据
20     this.range[0] = pList
21     this.range[1] = cList
22     this.range[2] = aList
23     this.values = ['广东省', '深圳市', '罗湖区']
24   })

```

Fence 19

今日核心:

- async 函数、Promise静态方法

1. [图片素材.zip](#)

# async函数和 await

Promise 虽然不用嵌套了，但是依旧有回调函数，是否可以更进一步？可以的，用 async 函数

## 1. 核心用法

:::warning

概念：

async/await是一种用于处理异步操作的Promise语法糖，使得编写异步代码变得更加简单和易读。通过使用async关键字声明一个函数为异步函数，并使用await关键字等待Promise的解析（完成或拒绝），以同步的方式编写异步操作的代码。

:::

简化之后的代码写出来大概这样

:::warning

核心步骤：

1. async 修饰函数
2. await 等待成功（Promise 对象）

:::

```
1  async function func() {
2    // await 获取到的是 之后 Promise 对象的成功结果
3    const res1 = await Promise对象1
4    const res2 = await Promise对象2
5    const res3 = await Promise对象3
6  }
7  func()
```

Fence 20

先写一个简单的例子

```
1  function randomNum(delay: number): Promise<string> {
2    return new Promise<string>((resolve, reject) => {
3      const num = Math.floor(Math.random() * 100)
4      resolve(num.toString())
5    })
6  }
7
8  //
9  async function func() {
10   const num1 = await randomNum(1000)
11   console.log(num1)
12   const num2 = await randomNum(1000)
13   console.log(num2)
14   const num3 = await randomNum(1000)
15   console.log(num3)
16   const num4 = await randomNum(1000)
17   console.log(num4)
18 }
19
20 func()
```

Fence 21

:::success

### 试一试：

1. 用刚刚学习的语法，改写【聊天机器人案例】
  2. 用刚刚学习的语法，改写【获取省市数据】的案例
- ...

```
1 import http from '@ohos.net.http';
2
3 interface ProResponse {
4     message: string
5     list: string[]
6 }
7
8 @Entry
9 @Component
10 struct Page05_CallBacks {
11     @State message: string = 'Hello World';
12     apfruits: string[] = ['apple1', 'apple2', 'apple3', 'apple4']
13     orfruits: string[] = ['orange1', 'orange2', 'orange3', 'orange4']
14     pefruits: string[] = ['peach1', 'peach2', 'peach3', 'peach4']
15     // 二维数组，数组的每一项都是数组
16     @State multi: string[][] = [this.apfruits, this.orfruits, this.pefruits]
17
18     // 回调函数嵌套的写法（可读性不好，不推荐）
19     func1() {
20         const req = http.createHttp()
21         // 1. 获取所有的省
22         req.request('https://hmajax.itheima.net/api/province')
23             .then(res => {
24                 // AlertDialog.show({
25                 //     message: res.result.toString()
26                 // })
27                 const proRes = JSON.parse(res.result.toString()) as ProResponse
28                 this.multi[0] = proRes.list // 省
29                 // 2. 获取某一个省下面的市 二维数组 this.multi[0] 省数组 this.multi[0]
[10] 省数组中的第 11 个
30                 req.request('https://hmajax.itheima.net/api/city?pname=' +
encodeURIComponent(this.multi[0][0]))// 北京
31                 .then(res => {
32                     // AlertDialog.show({
33                     //     message: res.result.toString()
34                     // })
35                     const cityRes = JSON.parse(res.result.toString()) as
ProResponse
36                     this.multi[1] = cityRes.list // 设置市
37
38                     // 3. 获取某一个市区下面的区
39                     // 省
40                     const pname = encodeURIComponent(this.multi[0][0]) // 省 北京
41                     // 市
42                     const cname = encodeURIComponent(this.multi[1][0]) // 市 对应 北
京市
43                     req.request(`https://hmajax.itheima.net/api/area?
pname=${pname}&cname=${cname}`)
```

```

44         .then(res => {
45             // AlertDialog.show({
46             //     message: res.result.toString()
47             // })
48             const areaRes = JSON.parse(res.result.toString()) as
ProResponse
49             this.multi[2] = areaRes.list
50         })
51     })
52
53
54     })
55 }
56
57 // 一打开就获取数据
58 aboutToAppear(): void {
59     const req = http.createHttp()
60     req.request('https://hmajax.itheima.net/api/province')
61     .then(res => {
62         const proRes = JSON.parse(res.result.toString()) as ProResponse
63         this.multi[0] = proRes.list // 省
64
65         // 返回一个新的 Promise 对象
66         return req.request('https://hmajax.itheima.net/api/city?pname=' +
encodeURIComponent(this.multi[0][0]))
67     })
68     .then(res => {
69         // AlertDialog.show({
70         //     message: res.result.toString()
71         // })
72         const cityRes = JSON.parse(res.result.toString()) as ProResponse
73         this.multi[1] = cityRes.list
74         // 获取 省
75         const pname = encodeURIComponent(this.multi[0][0])
76         // 市
77         const cname = encodeURIComponent(this.multi[1][0])
78
79         return req.request(`https://hmajax.itheima.net/api/area?
pname=${pname}&cname=${cname}`)
80     })// 等省
81     .then(res => {
82         // AlertDialog.show({
83         //     message: res.result.toString()
84         // })
85         const areaRes = JSON.parse(res.result.toString()) as ProResponse
86         this.multi[2] = areaRes.list
87     }) // 等市
88
89 }
90
91 build() {
92     column() {
93         TextPicker({ range: this.multi })
94         .canLoop(false) // 不能无限滚动
95         Button('修改数组')

```

```

96         .onClick(() => {
97             this.multi[0] = ['西瓜', '菠萝']
98         })
99     }
100     .height('100%')
101 }
102 }

```

Fence 22

## 2. 捕获错误

使用 `async` 之后需要 `try/catch` 来捕获异常

```

1  try {
2      // 需要被执行的语句
3  } catch (error) {
4      // error 接收错误信息
5      // try 有错误时执行的语句
6  }
7

```

Fence 23

:::success

**注意:**

`try-catch` 可以用来捕获任意的异常，而不仅仅局限于 `async` 函数

:::

咱们来测试一下

```

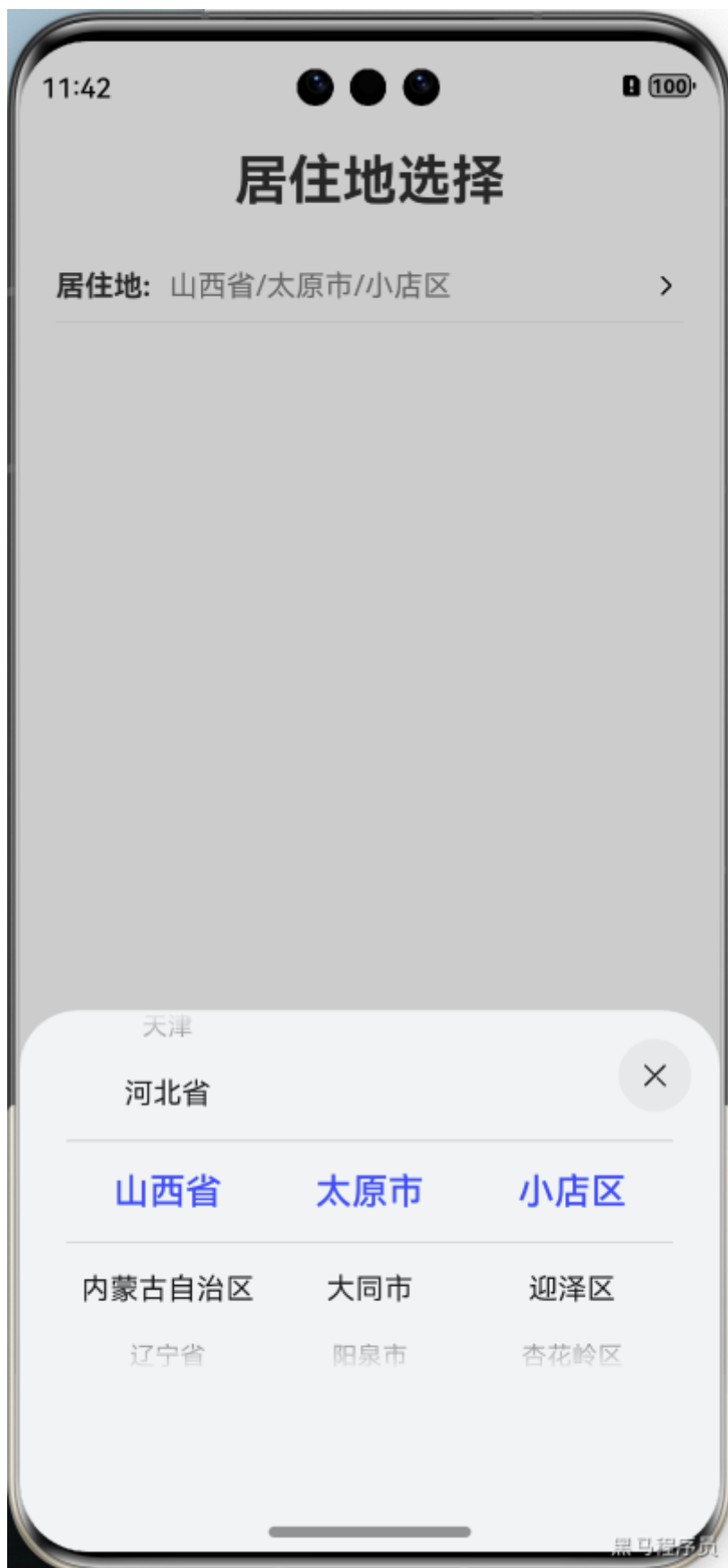
1  function randomNum(delay: number): Promise<string> {
2      return new Promise<string>((resolve, reject) => {
3          const num = Math.floor(Math.random() * 100)
4          if (num < 50) {
5              resolve(num.toString())
6          } else {
7              reject(num.toString())
8          }
9      })
10 }
11
12 async function func() {
13     try {
14         const num1 = await randomNum(1000)
15         console.log('func-res', num1) // 正常执行 打印 num1
16     } catch (error) {
17         console.log('func-error', error) // 出现错误 打印 error
18     }
19 }
20
21 func()

```

Fence 24

# 案例-省市区选择

使用刚刚学习的 `async` 和 `await` 咱们来完成省市区选择这个例子



```
1 import http from '@ohos.net.http';  
2
```

```

3 interface IResponse {
4     message: string
5     list: string[]
6 }
7
8
9 @Entry
10 @Component
11 struct Day03_06_AreaChange {
12     @State message: string = '居住地选择';
13     @State range: string[][] = [['北京'], ['北京市'], ['东城区']]
14     @State selected: number[] = [0, 0, 0]
15     // 这部分信息的目的是渲染到页面上
16     @State values: string[] = ['北京', '北京市', '东城区']
17     // 请求对象
18     req: http.HttpRequest = http.createHttp()
19     @State showSheet: boolean = false
20
21
22
23     aboutToAppear(): void {
24     }
25
26     build() {
27         Column({ space: 10 }) {
28             // 标题
29             Text(this.message)
30                 .fontSize(30)
31                 .fontWeight(FontWeight.Bold)
32                 .textAlign(TextAlign.Center)
33                 .width('100%')
34                 .margin({ bottom: 20 })
35             //
36             Row({ space: 10 }) {
37                 Text('居住地:')
38                     .fontWeight(FontWeight.Bold)
39                 Text(this.values.join('/'))
40                     .layoutWeight(1)
41                     .fontColor(Color.Gray)
42                     .onClick(() => {
43                         this.showSheet = true
44                     })
45                 Image($r('app.media.ic_public_arrow_right'))
46                     .width(20)
47             }
48
49             Divider()
50             Blank()
51
52
53         }
54         .height('100%')
55         .width('100%')
56         .alignItems(HorizontalAlign.Start)
57         .padding(20)

```

```

58     .bindSheet($$this.showSheet, this.areaSheet(), {
59         height: 300
60     })
61 }
62
63 @Builder
64 areaSheet() {
65     Column() {
66         TextPicker({
67             range: this.range,
68             selected: $$this.selected,
69             // value: $$this.values //
70         })
71         .canLoop(false)
72     }
73 }
74 }

```

Fence 25

## 1. 默认省市区

默认展示北京的信息



:::success

步骤:

1. aboutToAppear, 获取 所有省, 第一个省的市, 第一个市的区

:::

```

1 import http from '@ohos.net.http';
2

```

```

3 interface IResponse {
4     message: string
5     list: string[]
6 }
7
8
9 @Entry
10 @Component
11 struct Day03_06_AreaChange {
12     // 标题
13     @State message: string = '居住地选择';
14     // 省市区数组
15     @State range: string[][] = [['北京'], ['北京市'], ['东城区']]
16     // 选中的索引
17     @State selected: number[] = [0, 0, 0]
18     // 这部分信息的目的是渲染到页面上
19     @State values: string[] = ['北京', '北京市', '东城区']
20     // 请求对象
21     req: http.HttpRequest = http.createHttp()
22     // 是否显示半模态
23     @State showSheet: boolean = false
24
25
26
27 build() {
28     Column({ space: 10 }) {
29         // 标题
30         Text(this.message)
31             .fontSize(30)
32             .fontWeight(FontWeight.Bold)
33             .textAlign(TextAlign.Center)
34             .width('100%')
35             .margin({ bottom: 20 })
36         //
37         Row({ space: 10 }) {
38             Text('居住地:')
39                 .fontWeight(FontWeight.Bold)
40             Text(this.values.join('/'))
41                 .layoutWeight(1)
42                 .fontColor(Color.Gray)
43                 .onClick(() => {
44                     this.showSheet = true
45                 })
46             Image($r('app.media.ic_public_arrow_right'))
47                 .width(20)
48         }
49
50         Divider()
51         Blank()
52
53
54     }
55     .height('100%')
56     .width('100%')
57     .alignItems(HorizontalAlign.Start)

```

```

58     .padding(20)
59     .bindSheet($$this.showSheet, this.areaSheet(), {
60         height: 300
61     })
62 }
63
64 @Builder
65 areaSheet() {
66     column() {
67         TextPicker({
68             range: this.range,
69             selected: $$this.selected,
70         })
71         .canLoop(false)
72     }
73 }
74 }

```

Fence 26

## 2. 省市区联动效果-省

先来完成省份改变时，重新获取对应的市以及对应的地区的效果

[传送门](#)

:::success

**需求：**

1. 选中不同的省之后，获取对应的市和区

**分析：**

1. 如何知道选中了不同的省？
  1. 找对应的事件-onChange
2. 如果市和区不是第一个，重新选中省之后需要如何处理？
  1. 市和区要还原为第一个
3. 如何获取上一次选择的【省市区】？
  1. 有一个状态变量 values 把最新选中的值保存进去
  2. onChange 触发的时候，判断 values 的值，和当前选中的值是否相同即可

```

1  import http from '@ohos.net.http';
2
3  interface IResponse {
4      message: string
5      list: string[]
6  }
7
8
9  @Entry
10 @Component
11 struct Day03_06_AreaChange {
12     @State message: string = '居住地选择';
13     @State range: string[][] = [['北京'], ['北京市'], ['东城区']]
14     @State selected: number[] = [0, 0, 0]

```

```

15 // values中设置的内容必须 在 range 中存在, 否则会崩溃
16 @State values: string[] = ['北京', '北京市', '东城区']
17 // 请求对象
18 req: http.HttpRequest = http.createHttp()
19 @State showSheet: boolean = false
20 // 定时器 id
21 timeId: number = -1
22
23 // 获取省市区数据
24 async getAllData() {
25     const res1 = await
this.req.request('https://hmajax.itheima.net/api/province')
26     const proRes = JSON.parse(res1.result.toString()) as IResponse
27
28     // 获取第一个省对应的城市
29     const res2 = await
this.req.request('https://hmajax.itheima.net/api/city?pname=' +
encodeURIComponent(proRes.list[0]))
30
31     const cityRes = JSON.parse(res2.result.toString()) as IResponse
32
33     // url?key=value&key2=value2 格式不要写错啦
34     const res3 = await
this.req.request(`https://hmajax.itheima.net/api/area?
pname=${encodeURIComponent(proRes.list[0])}&cname=${encodeURIComponent(city
Res.list[0])}`)
35     const areaRes = JSON.parse(res3.result.toString()) as IResponse
36     this.range[0] = proRes.list
37     this.range[1] = cityRes.list
38     this.range[2] = areaRes.list
39 }
40
41 // 链式编程写法
42 aboutToAppear(): void {
43     this.getAllData()
44 }
45
46 build() {
47     column({ space: 10 }) {
48         Text(this.message)
49             .fontSize(30)
50             .fontWeight(FontWeight.Bold)
51             .textAlign(TextAlign.Center)
52             .width('100%')
53             .margin({ bottom: 20 })
54         Row({ space: 10 }) {
55             Text('居住地:')
56                 .fontWeight(FontWeight.Bold)
57             Text(this.values.join('/'))
58                 .layoutWeight(1)
59                 .fontColor(Color.Gray)
60                 .onClick(() => {
61                     this.showSheet = true
62                 })
63             Image($r('app.media.ic_public_arrow_right'))

```

```

64         .width(20)
65     }
66
67     Divider()
68     Blank()
69
70
71 }
72 .height('100%')
73 .width('100%')
74 .alignItems(HorizontalAlign.Start)
75 .padding(20)
76 .bindSheet($$this.showSheet, this.areaSheet(), {
77     height: 300
78 })
79 }
80
81 @Builder
82 areaSheet() {
83     column() {
84         TextPicker({
85             range: this.range,
86             selected: $$this.selected,
87             // value: $$this.values //
88         })
89         .canLoop(false)
90         .onChange(async (value, index) => {
91             // 获取最新选中的省市区
92             const _pname = value[0]
93             const res2 = await
this.req.request('https://hmajax.itheima.net/api/city?pname=' +
encodeURIComponent(_pname))
94             const cityRes = JSON.parse(res2.result.toString()) as IResponse
95             this.range[1] = cityRes.list
96             this.values[0] = _pname
97             this.selected[1] = 0
98             // 市区应该是第一个
99             this.values[1] = cityRes.list[0]
100             // 基于最新的市区 获取 对应的地区
101             const res3 = await
this.req.request(`https://hmajax.itheima.net/api/area?
pname=${encodeURIComponent(this.values[0])}&cname=${encodeURIComponent(this
.values[1])}`)
102             const areaRes = JSON.parse(res3.result.toString()) as IResponse
103             this.range[2] = areaRes.list
104             // 默认选中第一个
105             this.selected[2] = 0
106             // 保存最新的地区
107             this.values[2] = areaRes.list[0]
108         })
109     }
110 }
111 }

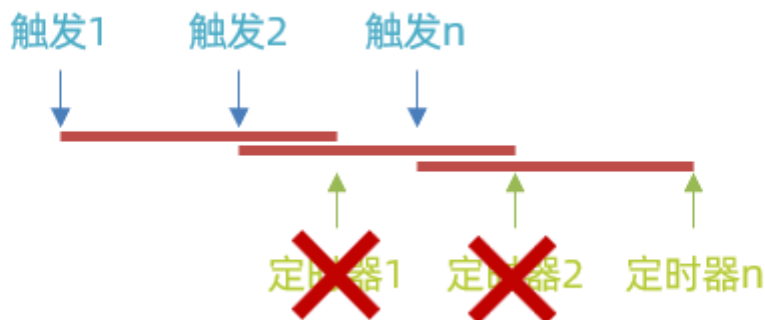
```

### 3. 函数防抖

常见的性能优化方案，它可以防止高频渲染页面时出现的视觉抖动(卡顿):比如上一节写的例子里面就会有抖动效果，这里可以使用函数防抖来解决

防抖的效果:

1. 连续事件停止触发后，一段时间内如果没有再次触发，就执行业务代码



黑马程序员

核心步骤:

1. 开启定时器，并保存定时器 ID
2. 清除已开启的定时器

```
1 let timeId
2 clearTimeout(timeId)
3 timeId = setTimeout(()=>{
4   // 业务代码
5 })
```

Fence 28

使用防抖改写刚刚的代码

```
1 import http from '@ohos.net.http';
2
3 interface IResponse {
4   message: string
5   list: string[]
6 }
7
8
9 @Entry
10 @Component
11 struct Day03_06_AreaChange {
12   @State message: string = '居住地选择';
13   @State range: string[][] = [['北京'], ['北京市'], ['东城区']]
14   @State selected: number[] = [0, 0, 0]
15   // values中设置的内容必须 在 range 中存在，否则会崩溃
16   @State values: string[] = ['北京', '北京市', '东城区']
17   // 请求对象
18   req: http.HttpRequest = http.createHttp()
19   @State showSheet: boolean = false
20   // 定时器 id
```

```

21     timeId: number = -1
22
23     // 获取省市数据
24     async getAllData() {
25         const res1 = await
this.req.request('https://hmajax.itheima.net/api/province')
26         const proRes = JSON.parse(res1.result.toString()) as IResponse
27
28         // 获取第一个省对应的城市
29         const res2 = await
this.req.request('https://hmajax.itheima.net/api/city?pname=' +
encodeURIComponent(proRes.list[0]))
30
31         const cityRes = JSON.parse(res2.result.toString()) as IResponse
32
33         // url?key=value&key2=value2 格式不要写错啦
34         const res3 = await
this.req.request(`https://hmajax.itheima.net/api/area?
pname=${encodeURIComponent(proRes.list[0])}&cname=${encodeURIComponent(city
Res.list[0])}`)
35         const areaRes = JSON.parse(res3.result.toString()) as IResponse
36         this.range[0] = proRes.list
37         this.range[1] = cityRes.list
38         this.range[2] = areaRes.list
39     }
40
41     // 链式编程写法
42     aboutToAppear(): void {
43         this.getAllData()
44     }
45
46     build() {
47         Column({ space: 10 }) {
48             Text(this.message)
49                 .fontSize(30)
50                 .fontWeight(FontWeight.Bold)
51                 .textAlign(TextAlign.Center)
52                 .width('100%')
53                 .margin({ bottom: 20 })
54             Row({ space: 10 }) {
55                 Text('居住地:')
56                     .fontWeight(FontWeight.Bold)
57                 Text(this.values.join('/'))
58                     .layoutWeight(1)
59                     .fontColor(Color.Gray)
60                     .onClick(() => {
61                         this.showSheet = true
62                     })
63                 Image($r('app.media.ic_public_arrow_right'))
64                     .width(20)
65             }
66
67             Divider()
68             Blank()
69

```

```

70
71     }
72     .height('100%')
73     .width('100%')
74     .alignItems(HorizontalAlign.Start)
75     .padding(20)
76     .bindSheet($$this.showSheet, this.areaSheet(), {
77         height: 300
78     })
79 }
80
81 @Builder
82 areaSheet() {
83     column() {
84         TextPicker({
85             range: this.range,
86             selected: $$this.selected,
87             // value: $$this.values //
88         })
89         .canLoop(false)
90         .onChange(async (value, index) => {
91             clearTimeout(this.timeId)
92             this.timeId = setTimeout(async () => {
93                 // 获取最新选中的省市区
94                 const _pname = value[0]
95                 const res2 = await
this.req.request('https://hmajax.itheima.net/api/city?pname=' +
encodeURIComponent(_pname))
96                 const cityRes = JSON.parse(res2.result.toString()) as IResponse
97                 this.range[1] = cityRes.list
98                 this.values[0] = _pname
99                 this.selected[1] = 0
100                 // 市区应该是第一个
101                 this.values[1] = cityRes.list[0]
102                 // 基于最新的市区 获取 对应的地区
103                 const res3 = await
this.req.request(`https://hmajax.itheima.net/api/area?
pname=${encodeURIComponent(this.values[0])}&cname=${encodeURIComponent(this
.values[1])}`)
104                 const areaRes = JSON.parse(res3.result.toString()) as IResponse
105                 this.range[2] = areaRes.list
106                 // 默认选中第一个
107                 this.selected[2] = 0
108                 // 保存最新的地区
109                 this.values[2] = areaRes.list[0]
110             }, 300)
111         })
112     }
113 }
114 }
115 }

```

## 4. 省市区联动效果-市

接下来完成市的联动效果

:::warning

核心步骤:

1. 改变了市：区重新获取，并选择 0
  1. 上一次的市和最新选中的市比较
    1. 改变了，重新获取区，选择 0
2. 改变了区：只需要更改页面显示
  1. 修改渲染数组的项即可
  - ...

```
1  import http from '@ohos.net.http';
2
3  interface IResponse {
4      message: string
5      list: string[]
6  }
7
8
9  @Entry
10 @Component
11 struct Day03_06_AreaChange {
12     @State message: string = '居住地选择';
13     @State range: string[][] = [['北京'], ['北京市'], ['东城区']]
14     @State selected: number[] = [0, 0, 0]
15     // values中设置的内容必须在 range 中存在，否则会崩溃
16     @State values: string[] = ['北京', '北京市', '东城区']
17     // 请求对象
18     req: http.HttpRequest = http.createHttp()
19     @State showsheet: boolean = false
20     // 定时器 id
21     timeId: number = -1
22
23     // 获取省市区数据
24     async getAllData() {
25         const res1 = await
26         this.req.request('https://hmajax.itheima.net/api/province')
27         const proRes = JSON.parse(res1.result.toString()) as IResponse
28
29         // 获取第一个省对应的城市
30         const res2 = await
31         this.req.request('https://hmajax.itheima.net/api/city?pname=' +
32         encodeURIComponent(proRes.list[0]))
33
34         const cityRes = JSON.parse(res2.result.toString()) as IResponse
35
36         // url?key=value&key2=value2 格式不要写错啦
37         const res3 = await
38         this.req.request(`https://hmajax.itheima.net/api/area?
39         pname=${encodeURIComponent(proRes.list[0])}&cname=${encodeURIComponent(city
40         Res.list[0])}`)
41         const areaRes = JSON.parse(res3.result.toString()) as IResponse
42         this.range[0] = proRes.list
```

```

37     this.range[1] = cityRes.list
38     this.range[2] = areaRes.list
39 }
40
41 // 链式编程写法
42 aboutToAppear(): void {
43     this.getAllData()
44 }
45
46 build() {
47     Column({ space: 10 }) {
48         Text(this.message)
49             .fontSize(30)
50             .fontWeight(FontWeight.Bold)
51             .textAlign(TextAlign.Center)
52             .width('100%')
53             .margin({ bottom: 20 })
54         Row({ space: 10 }) {
55             Text('居住地:')
56                 .fontWeight(FontWeight.Bold)
57             Text(this.values.join('/'))
58                 .layoutWeight(1)
59                 .fontColor(Color.Gray)
60                 .onClick(() => {
61                     this.showSheet = true
62                 })
63             Image($r('app.media.ic_public_arrow_right'))
64                 .width(20)
65         }
66
67         Divider()
68         Blank()
69
70     }
71
72     .height('100%')
73     .width('100%')
74     .alignItems(HorizontalAlign.Start)
75     .padding(20)
76     .bindSheet($$this.showSheet, this.areaSheet(), {
77         height: 300
78     })
79 }
80
81 @Builder
82 areaSheet() {
83     Column() {
84         TextPicker({
85             range: this.range,
86             selected: $$this.selected,
87             // value: $$this.values //
88         })
89         .canLoop(false)
90         .onChange(async (value, index) => {
91             // this.values = value as string[]

```

```

92         // 获取最新选中的省市区
93         const _pname = value[0]
94         const _cname = value[1]
95         const _aname = value[2]
96         // 基于省份 获取最新的市
97         clearTimeout(this.timeId)
98         // 没有开启定时器 开启定时器 并保存 id
99         this.timeId = setTimeout(async () => {
100
101             // 基于省 获取最新的市
102             // 如果省 发生改变, 那么重新获取对应的市区 并将选中值设置为 0
103             if (_pname !== this.values[0]) {
104                 const res2 = await
this.req.request('https://hmajax.itheima.net/api/city?pname=' +
encodeURIComponent(_pname))
105                 const cityRes = JSON.parse(res2.result.toString()) as
IResponse
106                 this.range[1] = cityRes.list
107                 this.values[0] = _pname
108                 this.selected[1] = 0
109                 // 市区应该是第一个
110                 this.values[1] = cityRes.list[0]
111                 // 基于最新的市区 获取 对应的地区
112                 const res3 = await
this.req.request(`https://hmajax.itheima.net/api/area?
pname=${encodeURIComponent(this.values[0])}&cname=${encodeURIComponent(this
.values[1])}`)
113                 const areaRes = JSON.parse(res3.result.toString()) as
IResponse
114                 this.range[2] = areaRes.list
115                 // 默认选中第一个
116                 this.selected[2] = 0
117                 // 保存最新的地区
118                 this.values[2] = areaRes.list[0]
119                 return
120             }
121
122             // 省份没变, 但是 市变了
123             if (_cname !== this.values[1]) {
124                 const res3 = await
this.req.request(`https://hmajax.itheima.net/api/area?
pname=${encodeURIComponent(this.values[0])}&cname=${encodeURIComponent(_cna
me)}`)
125                 const areaRes = JSON.parse(res3.result.toString()) as
IResponse
126                 this.range[2] = areaRes.list
127                 this.values[1] = _cname
128                 // 默认选中第一个
129                 this.selected[2] = 0
130                 // 保存最新的地区
131                 this.values[2] = areaRes.list[0]
132                 return
133             }
134
135             this.values[2] = _aname

```

```
136
137         }, 200)
138     })
139 }
140 }
141 }
```

Fence 30

# Promise 的静态方法

日常开发中除了使用 Promise 对象以外，还可以通过 Promise 提供的静态方法来管理多个异步

Promise 的静态方法,其中较为常见的是以下 2 个

```
1 Promise.all() // 多个Promise, 全部成功, 某一个失败
2 Promise.race() // 多个 Promise, 第一个成功 or 失败
3 Promise.resolve() // 成功状态的 Promise 对象
4 Promise.reject() // 失败状态的 Promise 对象
```

Fence 31

首先是 resolve 和 reject 这两个在封装的时候见得多一些，不需要实例化，直接就可以获取 成功 或者 拒绝 的 Promise 对象,咱们之后看到之后知道是什么即可

## 1. Promise.resolve

返回一个成功原因的 Promise 对象

```
1 Promise.resolve('成功原因')
2   .then(res => {
3     AlertDialog.show({ message: res })
4   })
```

Fence 32

## 2. Promise.reject

返回一个拒绝原因的 Promise 对象

```
1 Promise.reject('拒绝原因')
2   .catch((err: string) => {
3     AlertDialog.show({ message: err })
4   })
```

Fence 33

## 3. Promise.race

传入 Promise 数组，第一个成功或者失败

```
1 const p1 = new Promise<string>((resolve, reject) => {
2   setTimeout(() => {
3     resolve('1')
4   }, 2000)
5 })
6 const p2 = new Promise<string>((resolve, reject) => {
7   setTimeout(() => {
8     reject('2')
9   }, 1000)
10 })
11 Promise.race([p1, p2, 'itheima']).then((res) => {
12   console.log('res:', res)
13 }, (err:string) => {
14   console.log('err:', err)
```

:::warning

上面 3 个方法日常开发中偶尔会出现，了解即可

:::

## 4. Promise.all

这个用的最多，多个 Promise，全部成功，或者第一个失败

首先通过一个简单的代码看看如何执行

```
1  const p1 = new Promise<string>((resolve, reject) => {
2    setTimeout(() => {
3      resolve('1')
4    }, 2000)
5  })
6  const p2 = new Promise<string>((resolve, reject) => {
7    setTimeout(() => {
8      reject('2')
9    }, 1000)
10 })
11 Promise.all([p1, p2, 'itheima'])
12   .then((res) => {
13     console.log('res:', res)
14   }, (err: string) => {
15     console.log('err:', err)
16   })
```

来做一个小例子,获取 3 个分类的数据，全部获取到之后，打印出来

```
1  const req = http.createHttp()
2  req.request('https://hmajax.itheima.net/api/category/sub?id=1005000')
3  req.request('https://hmajax.itheima.net/api/category/sub?id=1005002')
4  req.request('https://hmajax.itheima.net/api/category/sub?id=1010000')
```

```
1  async function func(){
2    const req = http.createHttp()
3    const p1 = req.request('https://hmajax.itheima.net/api/category/sub?
id=1005000')
4    const p2 = req.request('https://hmajax.itheima.net/api/category/sub?
id=1005002')
5    const p3 = req.request('https://hmajax.itheima.net/api/category/sub?
id=1010000')
6
7    // 结果是一个数组
8    const res = await Promise.all([p1, p2, p3])
9
10   // 我们要的是每一项的 result
11   const resList = res.map((v => v.result))
12
13   // 打印
```

```
14     AlertDialog.show({  
15         message: JSON.stringify(resList)  
16     })  
17 }
```

Fence 37

## 5. 案例-分类信息

---



基于刚刚学些 Promise.all来完成上述案例

:::warning

核心步骤:

1. 获取顶级分类

1. 获取顶级分类, children 中有 id 顶级分类 id

2. 基于顶级分类, 生成 Promise 数组获取二级分类

1. 每有一个顶级分类 id, 就需要调用一次 二级分类接口

2. 基于顶级分类 id, Promise 的数组
3. All管理, 获取到之后 渲染
3. 解析并渲染二级分类数据
- ...

```
1  import http from '@ohos.net.http';
2
3  interface TopCategoryResponse {
4      message: string
5      data: CategoryData[]
6  }
7
8  interface CategoryResponse {
9      message: string
10     data: CategoryData
11 }
12
13 interface CategoryData {
14     id: string
15     name: string
16     picture: string
17     children?: CategoryData[]
18 }
19
20 @Entry
21 @Component
22 struct Day03_08_GetAllCategory {
23     build() {
24         column() {
25             // 头部-
26             this.headerBuilder()
27             // 内容
28             Tabs({ barPosition: BarPosition.Start }) {
29                 TabContent() {
30                     mainContent()
31                 }
32                 .backgroundColor('#f6f6f6')
33                 .tabBar('首页')
34
35                 this.tabItemBuilder('小米自营')
36                 this.tabItemBuilder('手机数码')
37                 this.tabItemBuilder('小米电视')
38                 this.tabItemBuilder('电脑办公')
39                 this.tabItemBuilder('大家电')
40                 this.tabItemBuilder('小家电')
41                 this.tabItemBuilder('美食酒饮')
42                 this.tabItemBuilder('家居家装')
43                 this.tabItemBuilder('日常元素')
44                 this.tabItemBuilder('服装配饰')
45                 this.tabItemBuilder('有品海购')
46                 this.tabItemBuilder('手表首饰')
47                 this.tabItemBuilder('出行车品')
48                 this.tabItemBuilder('美妆个护')
49
50             }
```

```

51     }
52     .vertical(true)
53     .barwidth(100)
54     .barHeight('100%')
55 }
56 .width('100%')
57 .height('100%')
58 }
59
60 @Builder
61 headerBuilder() {
62     // 头部-
63     Stack({ alignContent: Alignment.End }) {
64         Text('分类')
65         .width('100%')
66         .textAlign(TextAlign.Center)
67         .fontSize(20);
68         // Image($r('app.media.ic_public_search'))
69         // .width(25);
70     }
71     .backgroundColor(Color.white)
72     .padding(15)
73
74 }
75
76 @Builder
77 tabItemBuilder(title: string) {
78     TabContent() {
79         Text(title + '的内容')
80         .fontSize(30)
81     }
82     .backgroundColor('#f6f6f6')
83     .tabBar(title)
84 }
85 }
86
87 @Component
88 struct mainContent {
89     @State topCategoryList: CategoryData[] = [
90         {
91             "id": "1005000",
92             "name": "居家",
93             "picture": "http://yjy-xiaotuxian-dev.oss-cn-
beijing.aliyuncs.com/picture/2021-05-06/201516e3-25d0-48f5-bcee-
7f0cafb14176.png",
94             "children": [
95                 {
96                     "id": "1005009",
97                     "name": "茶咖酒具",
98                     "picture":
"https://yanxuan.nosdn.127.net/3102b963e7a3c74b9d2ae90e4380da65.png?
quality=95&imageView"
99                 },
100                 {
101                     "id": "1007000",

```

```

102         "name": "水具杯壶",
103         "picture":
104         "https://yanxuan.nosdn.127.net/45b50d5f8afb6fdef97314647dcb7db.png?
105         quality=95&imageView"
106     },
107     {
108         "id": "1017000",
109         "name": "宠物食品",
110         "picture":
111         "https://yanxuan.nosdn.127.net/b42a85ef15f856081ea9f49e5f6893e2.png?
112         quality=95&imageView"
113     },
114     {
115         "id": "109248004",
116         "name": "宠物用品",
117         "picture":
118         "https://yanxuan.nosdn.127.net/e766b09029ca00680d1e651b5cdc42bd.png?
119         quality=95&imageView"
120     }
121 ]
122
123 @State isLoading: boolean = true
124 req = http.createHttp()
125
126 build() {
127     scroll() {
128         if (this.isLoading) {
129             LoadingProgress()
130                 .width(150)
131                 .color('#ccc')
132         } else {
133             column() {
134                 Image($r('app.media.ic_public_category_cover'))
135                     .width('100%')
136                 List() {
137                     ForEach(this.topCategoryList, (item: CategoryData) => {
138                         ListItemGroup({ header: this.groupHeader(item.name,
139 item.picture) }) {
140                             ForEach(item.children, (it: CategoryData) => {
141                                 ListItem() {
142                                     column() {
143                                         Image(it.picture)// .width('80%')
144                                             .height(90)
145                                         Text(it.name)
146                                             .fontSize(15)
147                                     }
148                                     .width('100%')
149                                     .alignItems(HorizontalAlign.Center)
150                                 }
151                             })
152                         }
153                     })
154                 }
155                 .backgroundImagePosition({ x: '0', y: 0 })
156             })
157         }
158     }
159 }

```

```

150         }
151         .backgroundColor(Color.White)
152         .lanes(2)
153         .divider({ strokeWidth: 10, color: '#f6f6f6' })
154     }
155     .justifyContent(FlexAlign.Start)
156 }
157
158 }
159 .height('100%')
160 .padding(10)
161 }
162
163 @Builder
164 groupHeader(title: string, icon: string) {
165     Row({ space: 5 }) {
166         Text(title)
167         .fontWeight(FontWeight.Bold)
168         Image(icon)
169         .width(20)
170     }
171     .width('100%')
172     .padding(10)
173     .backgroundColor(Color.White)
174 }
175 }
176
177

```

Fence 38

```

1  import http from '@ohos.net.http';
2
3  interface TopCategoryResponse {
4      message: string
5      data: CategoryData[]
6  }
7
8  interface CategoryResponse {
9      message: string
10     data: CategoryData
11 }
12
13 interface CategoryData {
14     id: string
15     name: string
16     picture: string
17     children?: CategoryData[]
18 }
19
20 @Entry
21 @Component
22 struct Day03_08_GetAllCategory {
23     build() {
24         Column() {
25             // 头部-

```

```

26     this.headerBuilder()
27     // 内容
28     Tabs({ barPosition: BarPosition.Start }) {
29         TabContent() {
30             mainContent()
31         }
32         .backgroundColor('#f6f6f6')
33         .tabBar('首页')
34
35         this.tabItemBuilder('小米自营')
36         this.tabItemBuilder('手机数码')
37         this.tabItemBuilder('小米电视')
38         this.tabItemBuilder('电脑办公')
39         this.tabItemBuilder('大家电')
40         this.tabItemBuilder('小家电')
41         this.tabItemBuilder('美食酒饮')
42         this.tabItemBuilder('家居家装')
43         this.tabItemBuilder('日常元素')
44         this.tabItemBuilder('服装配饰')
45         this.tabItemBuilder('有品海购')
46         this.tabItemBuilder('手表首饰')
47         this.tabItemBuilder('出行车品')
48         this.tabItemBuilder('美妆个护')
49
50
51     }
52     .vertical(true)
53     .barWidth(100)
54     .barHeight('100%')
55 }
56 .width('100%')
57 .height('100%')
58 }
59
60 @Builder
61 headerBuilder() {
62     // 头部-
63     Stack({ alignContent: Alignment.End }) {
64         Text('分类')
65         .width('100%')
66         .textAlign(TextAlign.Center)
67         .fontSize(20);
68         Image($r('app.media.ic_public_search'))
69         .width(25);
70     }
71     .backgroundColor(Color.White)
72     .padding(15)
73
74 }
75
76 @Builder
77 tabItemBuilder(title: string) {
78     TabContent() {
79         Text(title + '的内容')
80         .fontSize(30)

```

```

81     }
82     .backgroundColor('#f6f6f6')
83     .tabBar(title)
84   }
85 }
86
87 @Component
88 struct mainContent {
89   @State topCategoryList: CategoryData[] = [
90     {
91       "id": "1005000",
92       "name": "居家",
93       "picture": "http://yjy-xiaotuxian-dev.oss-cn-
beijing.aliyuncs.com/picture/2021-05-06/201516e3-25d0-48f5-bcee-
7f0cafb14176.png",
94       "children": [
95         {
96           "id": "1005009",
97           "name": "茶咖酒具",
98           "picture":
"https://yanxuan.nosdn.127.net/3102b963e7a3c74b9d2ae90e4380da65.png?
quality=95&imageView"
99         },
100        {
101          "id": "1007000",
102          "name": "水具杯壶",
103          "picture":
"https://yanxuan.nosdn.127.net/45b50d5f8afb6fdef97314647dcb7db.png?
quality=95&imageView"
104        },
105        {
106          "id": "1017000",
107          "name": "宠物食品",
108          "picture":
"https://yanxuan.nosdn.127.net/b42a85ef15f856081ea9f49e5f6893e2.png?
quality=95&imageView"
109        },
110        {
111          "id": "109248004",
112          "name": "宠物用品",
113          "picture":
"https://yanxuan.nosdn.127.net/e766b09029ca00680d1e651b5cdc42bd.png?
quality=95&imageView"
114        }
115      ]
116    },
117  ]
118   @State isLoading: boolean = true
119   req = http.createHttp()
120
121   async getTopCategory() {
122     const res = await
this.req.request('https://h Ajax.itheima.net/api/category/top')
123     const topCategoryRes = JSON.parse(res.result.toString()) as
TopCategoryResponse

```

```

124
125     const proArray = topCategoryRes.data.map<Promise<http.HttpResponse>>
(((v: CategoryData) => {
126         return this.req.request(`https://hmajax.itheima.net/api/category/sub?
id=${v.id}`)
127     })))
128
129     const allCategoryRes = await Promise.all<http.HttpResponse>(proArray)
130     const resultArr = allCategoryRes.map((v: http.HttpResponse) => {
131         const eachResponse = JSON.parse(v.result.toString()) as
CategoryResponse
132         return eachResponse.data
133     })
134     this.topCategoryList = resultArr
135
136     this.isLoading = false
137 }
138
139 aboutToAppear(): void {
140     this.getTopCategory()
141 }
142
143 onPageHide(): void {
144     console.log('onPageHide')
145 }
146
147 build() {
148     scroll() {
149         if (this.isLoading) {
150             loadingProgress()
151                 .width(150)
152                 .color('#ccc')
153         } else {
154             column() {
155                 image($r('app.media.ic_public_category_cover'))
156                     .width('100%')
157                 list() {
158                     forEach(this.topCategoryList, (item: CategoryData) => {
159                         listItemGroup({ header: this.groupHeader(item.name,
item.picture) }) {
160                             forEach(item.children, (it: CategoryData) => {
161                                 listItem() {
162                                     column() {
163                                         image(it.picture)// .width('80%')
164                                             .height(90)
165                                         text(it.name)
166                                             .fontSize(15)
167                                     }
168                                     .width('100%')
169                                     .alignItems(HorizontalAlign.Center)
170                                 }
171                             })
172                         })
173                     }
174                     .backgroundImagePosition({ x: '0', y: 0 })

```

```

175         })
176     }
177     .backgroundColor(Color.White)
178     .lanes(2)
179     .divider({ strokeWidth: 10, color: '#f6f6f6' })
180 }
181 .justifyContent(FlexAlign.Start)
182 }
183
184 }
185 .height('100%')
186 .padding(10)
187 }
188
189 @Builder
190 groupHeader(title: string, icon: string) {
191     Row({ space: 5 }) {
192         Text(title)
193         .fontWeight(FontWeight.Bold)
194         Image(icon)
195         .width(20)
196     }
197     .width('100%')
198     .padding(10)
199     .backgroundColor(Color.White)
200 }
201 }
202
203

```